

Daary Backend – Final Exhaustive Technical Report

1. Project overview

This project is a Django-powered backend for a real-estate marketplace ecosystem. It contains a buyer-facing flow, a developer/admin flow, and a recommendation engine. The architecture follows a clean Django app structure with the general pattern: model, selector, service, serializer, view, URL, tests.

The system supports:

- OTP-based authentication using Akedly
- buyer account creation and profile updates
- property listing discovery and search
- favorites management
- buyer inquiries and agent notifications
- developer account registration and management
- team permissions and developer dashboard
- project management with media upload support
- lead capture and export
- dashboard analytics
- AI-driven property recommendations based on survey answers

The repository is organized under apps/ and a root Django project directory. This makes the code modular, maintainable, and easy to extend.

2. Root project and setup files

manage.py

Purpose: standard Django project entry point.

What it does:

- sets the Django settings module
- runs management commands such as migrate, runserver, shell, test, and seed commands
- is the main command runner for the project

conftest.py

Purpose: pytest configuration for the Django project.

What it does:

- prepares Django test settings
- makes it easier to reuse the API client in tests
- keeps tests consistent with project configuration

requirements.txt

Purpose: dependency list.

What it includes:

- Django
- Django REST Framework
- JWT support
- Celery
- testing tools
- environment/deployment utilities

docker-compose.yml

Purpose: container orchestration configuration.

What it does:

- defines the services for the backend and supporting infrastructure
- helps local environment setup and service orchestration

Dockerfile

Purpose: container image for the backend service.

What it does:

- installs dependencies
- copies project code
- configures the app startup command

pytest.ini

Purpose: configuration for pytest.

What it does:

- sets project-level test behavior
- controls how Django tests are discovered and run

db.sqlite3

Purpose: local development database.

What it does:

- stores local data in the default development environment

3. Django project package files

daary_backend/__init__.py

Purpose: package initializer.

daary_backend/asgi.py

Purpose: ASGI entry point.

What it does:

- configures the application for async server environments

daary_backend/wsgi.py

Purpose: WSGI entry point.

What it does:

- configures the app for traditional web servers

daary_backend/celery.py

Purpose: Celery application bootstrap.

What it does:

- creates the Celery instance for async tasks
- connects the app to the Django project settings

daary_backend/urls.py

Purpose: top-level router for the application.

What it includes:

- /health/
- /admin/
- /api/v1/
- /api/docs/

This file includes app routes like auth, properties, search, inquiries, and developer APIs.

daary_backend/settings/base.py

Purpose: central Django configuration.

Includes:

- installed apps
- custom user model
- middleware
- database setup
- DRF configuration
- JWT settings
- CORS settings
- logging
- Celery config
- OpenAPI docs
- Agedly and AI provider config

daary_backend/settings/dev.py

Purpose: development overrides.

What it configures:

- debug mode
- local SQLite database
- local CORS allowances
- console email backend
- development token lifetime

daary_backend/settings/prod.py

Purpose: production environment configuration.

What it does:

- keeps production-specific secrets and settings separate from development

daary_backend/settings/test.py

Purpose: testing configuration.

What it does:

- provides a test configuration optimized for Django testing runs

4. Shared common utilities and base infrastructure

apps/common/__init__.py

Purpose: package indicator.

apps/common/models.py

Main class: TimestampedModel

What it does:

- adds created_at and updated_at
- implements consistent timestamp behavior
- centralizes common model metadata

apps/common/validators.py

Functions:

- validate_egyptian_phone(value)
- validate_birthday(value)

What they do:

- standardize Egyptian phone rules
- reject malformed or implausible values
- ensure realistic age validation

apps/common/permissions.py

Class: IsOwner

Purpose:

- allow ownership-based access checks
- used when a user should access only their own objects

apps/common/pagination.py

Class: StandardPagination

Purpose:

- gives the API consistent pagination metadata
- returns page count and navigation fields

apps/common/renderers.py

Class: ApiRenderer

Purpose:

- ensures all API responses follow a consistent JSON schema

apps/common/exceptions.py

Function: custom_exception_handler

Purpose:

- normalize DRF exception responses into the project's response format

apps/common/akedly.py

Purpose: abstraction for Akedly OTP verification provider.

Important members:

- AkedlyApiError
- _get_credentials()
- _parse_akedly_response(response)
- get_challenge()
- send_otp()
- verify_otp()

What it does:

- reads settings for the OTP provider
- obtains challenge data
- sends OTP codes to numbers
- verifies them and handles provider response errors

5. users app

apps/users/__init__.py

Package initializer.

apps/users/admin.py

Purpose: Django admin registration for the custom user model.

apps/users/apps.py

Configuration for the users app.

apps/users/models.py

Main components:

- UserManager
- User

Details:

- custom auth user model
- phone-based username
- role information

- birthday and profile data
- inactive until OTP confirmation completes

Important logic:

- UserManager validates phone and password before save
- User uses phone as the unique identifier
- account activation is tied to successful OTP verification

apps/users/selectors.py

Functions:

- get_user_by_phone(phone)
- get_user_by_id(user_id)

Purpose:

- centralize user lookup logic
- support account and profile flows

apps/users/services.py

Function:

- update_user_profile(user, fields)

Purpose:

- update public user fields without allowing phone changes
- enforce validation before saving

apps/users/serializers.py

Classes:

- UserProfileSerializer
- UserUpdateSerializer

Purpose:

- return profile information
- validate updates to user profile data

apps/users/views.py

Class:

- MeView

Endpoints:

- GET /api/v1/account/me/
- PATCH /api/v1/account/me/

Purpose:

- return current profile info
- update current user profile

apps/users/urls.py

Purpose: exposes account endpoints.

apps/users/migrations/

Migration files for the custom user model and database schema.

apps/users/tests/

Files:

- test_models.py
- test_views.py

Purpose:

- validate user creation, profile updates, phone handling, and account endpoints

6. authentication app

apps/authentication/__init__.py

Package initializer.

apps/authentication/apps.py

Auth app config.

apps/authentication/serializers.py

Classes:

- PowSolutionSerializer
- SendOtpSerializer
- VerifyOtpSerializer
- AuthTokenSerializer

Purpose:

- validate OTP challenge and verification payloads
- handle auth token output

apps/authentication/services.py

Functions:

- get_akedly_challenge()
- send_akedly_otp(...)
- verify_akedly_otp_and_authenticate(...)

Purpose:

- integrate with Akedly
- validate user login flow
- create JWT tokens after successful verification

apps/authentication/views.py

Classes:

- OtpRateThrottle
- _handle_akedly_error(err)
- ChallengeView
- SendOtpView

- VerifyOtpView
- LogoutView

Purpose:

- challenge endpoint
- OTP send endpoint
- OTP verification endpoint
- logout endpoint with refresh token invalidation

apps/authentication/urls.py

Routes:

- challenge/
- send-otp/
- verify-otp/
- token/refresh/
- logout/

apps/authentication/tasks.py

Purpose: auth-related background tasks, if used in future extensions.

apps/authentication/tests/

Files:

- test_akedly_client.py
- test_auth.py
- test_views.py

Purpose:

- validate challenge creation, OTP sending, verification, token refresh, and logout

7. properties app

apps/properties/__init__.py

Package initializer.

apps/properties/admin.py

Purpose: admin configuration for property-related models.

Includes admin support for:

- PropertyImageInline
- PropertyAdmin
- AgentAdmin
- AmenityAdmin

apps/properties/apps.py

Properties app config.

apps/properties/models.py

Main models:

- PropertyType
- CompletionStatus
- Furnishing
- PaymentMethod
- Agent
- Amenity
- Property
- PropertyImage

Purpose:

- define metadata and listing model structure for the marketplace

What they do:

- Property stores the listing itself
- Agent stores the assigned real-estate agent
- Amenity defines optional property features
- PropertyImage stores the gallery for each listing

apps/properties/selectors.py

Functions:

- get_property_by_id(pk)
- get_properties_queryset(filters)

Purpose:

- centralize listing fetch logic
- support filter-based property queries

apps/properties/services.py

Function:

- increment_view_count(property, request)

Purpose:

- prevent duplicate view counting from the same user/IP during a short period

apps/properties/serializers.py

Classes:

- AgentSerializer
- PropertyImageSerializer
- PropertyListSerializer
- PropertyDetailSerializer

Purpose:

- convert model objects into API-ready data

apps/properties/views.py

Classes:

- PropertyListView
- PropertyDetailView

Endpoints:

- GET /api/v1/properties/
- GET /api/v1/properties//

Purpose:

- list all properties and return details for one selected property

apps/properties/tasks.py

Function:

- calculate_location_medians()

Purpose:

- compute summary metrics for price segmentation or ranking support

apps/properties/management/commands/seed_data.py

Purpose: seed demo data.

What it does:

- populate the database with initial sample data for local testing

apps/properties/management/commands/seed_properties.py

Purpose: bulk seed property records.

What it does:

- generate a large property dataset for testing display and search functionality

apps/properties/migrations/

Migration history for listing scheme changes.

apps/properties/tests/

Files:

- test_models.py
- test_views.py

Purpose:

- validate listing model behavior and property API endpoints

8. search app

apps/search/__init__.py

Package initializer.

apps/search/apps.py

Search app config.

apps/search/ranking.py

Functions:

- `_get_segment_median_price(location, property_type)`
- `compute_relevance_score(...)`
- `apply_ordering(...)`

Purpose:

- compute relevance and rank properties in a search result set

The ranking logic rewards:

- price competitiveness
- recency
- completeness
- location and amenity fit
- request matching

apps/search/services.py

Function:

- `search_properties(filters=None, ordering=None)`

Purpose:

- build the filtered search result set
- performs search and ranking logic
- supports Arabic and local language aliases in search inputs

apps/search/views.py

Class:

- `PropertySearchView`

Endpoint:

- `GET /api/v1/search/`

Purpose:

- accept filter parameters and return a ranked property result set

apps/search/tasks.py

Function:

- `refresh_price_medians()`

Purpose:

- update median pricing used for search ranking boosts

apps/search/tests/

File:

- `test_services.py`

Purpose:

- validate the search service, filters, and ranking behavior

9. favorites app

apps/favorites/__init__.py

Package initializer.

apps/favorites/admin.py

Purpose: admin for saved item records.

apps/favorites/apps.py

Favorites app configuration.

apps/favorites/models.py

Model:

- Favorite

Purpose:

- represent saved property relationships for users

apps/favorites/serializers.py

Class:

- FavoriteSerializer

Purpose:

- return favorite data in JSON form

apps/favorites/services.py

Function:

- toggle_favorite(user, property)

Purpose:

- add or remove a property from a user's favorites
- returns the current favorite state

apps/favorites/views.py

Classes:

- FavoriteListView
- FavoriteToggleView

Endpoints:

- GET /api/v1/favorites/
- POST /api/v1/favorites//toggle/

Purpose:

- list favorites and toggle a current property as favorite or not favorite

apps/favorites/urls.py

Purpose: route mapping for favorites.

apps/favorites/tests/

File:

- test_views.py

Purpose:

- validate user-specific favorites and toggle behavior

10. inquiries app

apps/inquiries/__init__.py

Package initializer.

apps/inquiries/admin.py

Purpose: admin configuration for inquiry records.

apps/inquiries/apps.py

Inquiry app config.

apps/inquiries/models.py

Main enums:

- InquiryType
- InquiryStatus
- ContactMethod

Main model:

- Inquiry

Purpose:

- record buyer interest in a property and details about the inquiry

apps/inquiries/selectors.py

Functions:

- get_user_inquiries(user)
- get_inquiry_by_id(pk, user=None)

Purpose:

- fetch inquiry records correctly by user or by ID

apps/inquiries/services.py

Functions:

- create_inquiry(data, user=None, client_ip=None)
- update_lead(inquiry, status=None, rating=None)

Purpose:

- validate and save inquiry data
- reject spam traffic or duplicate submissions
- update lead status and rating

apps/inquiries/tasks.py

Function:

- send_inquiry_notification(self, inquiry_id)

Purpose:

- send inquiry notification to the notified party asynchronously

apps/inquiries/serializers.py

Classes:

- InquiryCreateSerializer
- InquiryListSerializer
- InquiryDetailSerializer

Purpose:

- validate and shape inquiry payloads

apps/inquiries/views.py

Classes:

- InquiryCreateView
- MyInquiriesListView
- InquiryDetailView

Endpoints:

- POST /api/v1/inquiries/
- GET /api/v1/inquiries/my/
- GET /api/v1/inquiries/

Purpose:

- allow buyers to create and review inquiries

apps/inquiries/urls.py

Purpose: route mapping for inquiry endpoints.

apps/inquiries/tests/

File:

- test_views.py

Purpose:

- validate inquiry creation, validation, and retrieval behavior

11. developer_accounts app

apps/developer_accounts/__init__.py

Package initializer.

apps/developer_accounts/admin.py

Admin registration for developer-related models.

apps/developer_accounts/apps.py

Developer accounts app config.

apps/developer_accounts/authentication.py

Class:

- DeveloperTokenAuthentication

Purpose:

- custom auth strategy for developer endpoints

apps/developer_accounts/models.py

Models:

- Permission
- DeveloperAccount
- DeveloperUser

Purpose:

- define developer account and team structure

apps/developer_accounts/selectors.py

Function:

- get_team_members(account)

Purpose:

- fetch team members attached to an account

apps/developer_accounts/permissions.py

Classes:

- IsDeveloperAuthenticated
- HasDeveloperPermission

Purpose:

- restrict access to developer-authorized users and actions

apps/developer_accounts/serializers.py

Classes:

- PermissionSerializer
- LoginSerializer
- DeveloperUserSerializer
- AccountUpdateSerializer
- TeamMemberCreateSerializer
- TeamMemberSerializer

Purpose:

- validate developer credential data, team creation, and account updates

apps/developer_accounts/services.py

Functions:

- authenticate_developer(email, password)
- update_developer_account(user, data)
- invite_team_member(account, data)
- update_team_member(member, data)
- deactivate_team_member(member)
- delete_developer_account(user)

Purpose:

- centralize developer account lifecycle and team logic

apps/developer_accounts/views.py

Classes:

- LoginView
- RegisterView
- AccountView
- TeamListView
- TeamCreateView
- TeamDetailView

Endpoints:

- POST /api/v1/developer/login/
- POST /api/v1/developer/register/
- GET /api/v1/developer/account/
- PUT /api/v1/developer/account/
- DELETE /api/v1/developer/account/
- GET /api/v1/developer/team/
- POST /api/v1/developer/team/invite/
- PUT /api/v1/developer/team/
- DELETE /api/v1/developer/team/

apps/developer_accounts/urls.py

Purpose: route definitions for developer account management.

apps/developer_accounts/migrations/

Migration files for the account model and seeded permissions.

12. projects app

apps/projects/__init__.py

Package initializer.

apps/projects/admin.py

Purpose: project admin configuration and inline image support.

apps/projects/apps.py

Project app config.

apps/projects/models.py

Models:

- ProjectType
- ProjectStatus
- Project
- ProjectImage

Purpose:

- support project creation, status management, and media attachments

apps/projects/selectors.py

Functions:

- `get_project_by_id(pk, account)`
- `get_projects_queryset(account, filters)`

Purpose:

- fetch only projects belonging to the developer account

apps/projects/services.py

Functions:

- `create_project(account, data, files)`
- `update_project(project, data, files)`
- `delete_project(project)`

Purpose:

- create/update/delete project records and associated files

apps/projects/serializers.py

Classes:

- `ProjectImageSerializer`
- `ProjectListSerializer`
- `ProjectDetailSerializer`

Purpose:

- validate and return project payloads

apps/projects/views.py

Classes:

- `ProjectListCreateView`
- `ProjectDetailView`
- `ProjectStatsView`

Endpoints:

- `GET /api/v1/developer/projects/`
- `POST /api/v1/developer/projects/`
- `GET /api/v1/developer/projects/`
- `PUT /api/v1/developer/projects/`
- `DELETE /api/v1/developer/projects/`
- `GET /api/v1/developer/projects/stats/`

Purpose:

- allow full project lifecycle management in the developer panel

apps/projects/urls.py

Purpose: route definitions for developer projects.

13. locations app

apps/locations/admin.py

Admin registration for locations.

apps/locations/apps.py

Location app config.

apps/locations/models.py

Model:

- Location

Purpose:

- store geographic hierarchy and region names

apps/locations/serializers.py

Class:

- LocationSerializer

Purpose:

- return location data in a friendly shape

apps/locations/views.py

Class:

- LocationListView

Endpoint:

- GET /api/v1/locations/

Purpose:

- list available locations and support filtering by parent or search term

apps/locations/urls.py

Purpose: route mapping for locations.

14. marketing app

apps/marketing/apps.py

Marketing app config.

apps/marketing/models.py

Model:

- MarketingLead

Purpose:

- store incoming leads generated by campaigns or contact forms

apps/marketing/serializers.py

Class:

- MarketingLeadCreateSerializer

Purpose:

- validate marketing lead creation payloads

apps/marketing/views.py

Classes:

- MarketingLeadCreateView
- DeveloperLeadListView
- DeveloperLeadExportView

Endpoints:

- POST /api/v1/contact/lead/
- GET /api/v1/developer/leads/
- GET /api/v1/developer/leads/export/

Purpose:

- collect lead data externally
- let developers review and export them

apps/marketing/urls.py

Purpose: route definitions for marketing leads.

15. dashboard app

apps/dashboard/apps.py

Dashboard app config.

apps/dashboard/services.py

Function:

- get_dashboard_stats(account)

Purpose:

- produce summary analytics for a developer account

It may return:

- project totals
- active project counts
- lead totals
- latest counts
- source summaries
- location-based statistics

apps/dashboard/views.py

Class:

- DashboardOverviewView

Endpoint:

- GET /api/v1/developer/dashboard/overview/

Purpose:

- return dashboard metrics to the frontend

apps/dashboard/urls.py

Purpose: dashboard route registration.

16. ai app

apps/ai/__init__.py

Package initializer.

apps/ai/admin.py

Purpose: register survey-related models in Django admin.

apps/ai/apps.py

AI app config.

apps/ai/models.py

Models:

- SurveySession
- SurveyQuestion
- SurveyAnswer
- SurveyResult

Purpose:

- store the survey flow and final recommendation results

apps/ai/consultation_scoring.py

Functions:

- score_homebuyer_property(property_obj, answers)
- score_investor_property(property_obj, answers)
- score_property_for_track(property_obj, track, answers)

Purpose:

- compute property suitability score based on user answers and selected buying track

apps/ai/serializers.py

Classes:

- SurveyQuestionSerializer
- SurveyAnswerSerializer
- SurveyResultSerializer

Purpose:

- validate survey answers and serialize result data

apps/ai/services.py

Functions:

- `_ensure_question_bank()`
- `start_survey(user)`
- `_get_session_for_request(request, session_id)`
- `_serialize_matches(session)`
- `answer_survey_question(request, data)`

Purpose:

- build and manage the survey lifecycle
- save answers and compute matches against properties

apps/ai/views.py

Classes:

- `SurveyStartView`
- `SurveyAnswerView`
- `SurveyResultsView`

Endpoints:

- `POST /api/v1/ai/survey/start/`
- `POST /api/v1/ai/survey/answer/`
- `GET /api/v1/ai/survey//results/`

Purpose:

- start, answer, and retrieve the recommendation survey flow

apps/ai/urls.py

Purpose: survey route registration.

apps/ai/tests/

File:

- `test_survey.py`

Purpose:

- validate the AI survey logic and result generation

17. Full route map and controller mapping

Root app URLs

- `/health/` → health route
- `/admin/` → Django admin
- `/api/v1/` → main API namespace
- `/api/docs/` → API documentation output

Authentication APIs

- `GET /api/v1/auth/challenge/` → `ChallengeView`
- `POST /api/v1/auth/send-otp/` → `SendOtpView`

- POST /api/v1/auth/verify-otp/ → VerifyOtpView
- POST /api/v1/auth/token/refresh/ → Simple JWT refresh endpoint
- POST /api/v1/auth/logout/ → LogoutView

Buyer account APIs

- GET /api/v1/account/me/ → MeView
- PATCH /api/v1/account/me/ → MeView

Property APIs

- GET /api/v1/properties/ → PropertyListView
- GET /api/v1/properties// → PropertyDetailView

Search APIs

- GET /api/v1/search/ → PropertySearchView

Favorites APIs

- GET /api/v1/favorites/ → FavoriteListView
- POST /api/v1/favorites//toggle/ → FavoriteToggleView

Inquiry APIs

- POST /api/v1/inquiries/ → InquiryCreateView
- GET /api/v1/inquiries/my/ → MyInquiriesListView
- GET /api/v1/inquiries// → InquiryDetailView

Marketing APIs

- POST /api/v1/contact/lead/ → MarketingLeadCreateView
- GET /api/v1/developer/leads/ → DeveloperLeadListView
- GET /api/v1/developer/leads/export/ → DeveloperLeadExportView

Developer auth/account APIs

- POST /api/v1/developer/login/ → LoginView
- POST /api/v1/developer/register/ → RegisterView
- GET /api/v1/developer/account/ → AccountView
- PUT /api/v1/developer/account/ → AccountView
- DELETE /api/v1/developer/account/ → AccountView

Developer team APIs

- GET /api/v1/developer/team/ → TeamListView
- POST /api/v1/developer/team/invite/ → TeamCreateView
- PUT /api/v1/developer/team// → TeamDetailView
- DELETE /api/v1/developer/team// → TeamDetailView

Developer project APIs

- GET /api/v1/developer/projects/ → ProjectListCreateView
- POST /api/v1/developer/projects/ → ProjectListCreateView
- GET /api/v1/developer/projects// → ProjectDetailView

- PUT /api/v1/developer/projects// → ProjectDetailView
- DELETE /api/v1/developer/projects// → ProjectDetailView
- GET /api/v1/developer/projects//stats/ → ProjectStatsView

Dashboard APIs

- GET /api/v1/developer/dashboard/overview/ → DashboardOverviewView

AI survey APIs

- POST /api/v1/ai/survey/start/ → SurveyStartView
- POST /api/v1/ai/survey/answer/ → SurveyAnswerView
- GET /api/v1/ai/survey//results/ → SurveyResultsView

Location APIs

- GET /api/v1/locations/ → LocationListView

18. Exact request and response examples for all endpoints

18.1 Challenge request

Endpoint:

- GET /api/v1/auth/challenge/

Example request:

http

GET /api/v1/auth/challenge/

Authorization: none

Example response:

json

```
{
  "success": true,
  "data": {
    "challenge": "abc123",
    "nonce": 123456,
    "turnstile_enabled": false
  },
  "message": "Challenge generated successfully",
  "errors": null,
  "meta": {
    "request_id": "req_001"
  }
}
```

18.2 Send OTP request

Endpoint:

- POST /api/v1/auth/send-otp/

Example request:

json

```
{
  "phone_number": "01012345678",
  "pow_solution": {
    "challengeToken": "abc123",
    "nonce": 123456
  },
  "turnstile_token": "optional-token",
  "digits": 4
}
```

Example response:

json

```
{
  "success": true,
  "data": {
    "transaction_req_id": "tx_123",
    "phone_number": "01012345678",
    "sent": true
  },
  "message": "OTP sent successfully",
  "errors": null,
  "meta": {}
}
```

18.3 Verify OTP request

Endpoint:

- POST /api/v1/auth/verify-otp/

Example request:

json

```
{
  "transaction_req_id": "tx_123",
  "otp": "1234",
  "phone_number": "01012345678"
}
```

Example response:

json

```
{
```

```
"success": true,

"data": {

  "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {

    "id": 1,
    "phone": "01012345678",
    "name": "John Doe",
    "birthday": "1990-06-10",
    "role": "buyer",
    "is_active": true
  },
  "is_new_user": false
},
"message": "OTP verified successfully",
"errors": null,
"meta": {}
}
```

18.4 Refresh token request

Endpoint:

- POST /api/v1/auth/token/refresh/

Example request:

json

```
{
  "refresh": "eyJ...refreshtoken..."
}
```

Example response:

json

```
{
  "success": true,
  "data": {

    "access": "eyJ...newaccess..."
  },
  "message": "Token refreshed successfully",
  "errors": null,
  "meta": {}
}
```

18.5 Logout request

Endpoint:

- POST /api/v1/auth/logout/

Example request:

json

```
{  
  "refresh": "eyJ...refreshtoken..."  
}
```

Example response:

json

```
{  
  "success": true,  
  "data": {},  
  "message": "Logged out successfully",  
  "errors": null,  
  "meta": {}  
}
```

18.6 Get current user profile

Endpoint:

- GET /api/v1/account/me/

Example request:

http

GET /api/v1/account/me/

Authorization: Bearer

Example response:

json

```
{  
  "success": true,  
  "data": {  
    "id": 1,  
    "phone": "01012345678",  
    "name": "John Doe",  
    "birthday": "1990-06-10",  
    "role": "buyer",  
    "is_active": true  
  },  
  "message": "Profile loaded",  
  "errors": null,  
  "meta": {
```

```
"page": 1,  
"page_size": 20  
}  
}
```

18.7 Update user profile

Endpoint:

- PATCH /api/v1/account/me/

Example request:

json

```
{  
  "name": "John Updated",  
  "birthday": "1991-03-05"  
}
```

Example response:

json

```
{  
  "success": true,  
  "data": {  
    "id": 1,  
    "phone": "01012345678",  
    "name": "John Updated",  
    "birthday": "1991-03-05",  
    "role": "buyer"  
  },  
  "message": "Profile updated successfully",  
  "errors": null,  
  "meta": {}  
}
```

18.8 List properties

Endpoint:

- GET /api/v1/properties/

Example request:

http

GET /api/v1/properties/?location=nasr-city&property_type=apartment&min_price=500000&max_price=2000000

Authorization: Bearer

Example response:

json

```
{
  "success": true,
  "data": [
    {
      "id": 12,
      "title": "Luxury Apartment",
      "price": 1500000,
      "location": "Nasr City",
      "property_type": "apartment",
      "bedrooms": 2,
      "bathrooms": 2,
      "area_sqft": 120,
      "is_featured": true
    }
  ],
  "message": "Properties retrieved successfully",
  "errors": null,
  "meta": {
    "count": 1,
    "page": 1,
    "pages": 1,
    "next": null,
    "previous": null
  }
}
```

18.9 Get property detail

Endpoint:

- GET /api/v1/properties/

Example request:

http

GET /api/v1/properties/12/

Authorization: Bearer

Example response:

json

```
{
  "success": true,
  "data": {
    "id": 12,
```

```
"title": "Luxury Apartment",
"price": 1500000,
"description": "A spacious apartment in a prime location.",
"location": "Nasr City",
"property_type": "apartment",
"bedrooms": 2,
"bathrooms": 2,
"area_sqft": 120,
"amenities": ["parking", "security"],
"images": [
  {"id": 1, "image": "https://.../img1.jpg"}
],
"message": "Property details loaded",
"errors": null,
"meta": {}
}
```

18.10 Search properties

Endpoint:

- GET /api/v1/search/

Example request:

http

GET /api/v1/search/?location=nasr-city&bedrooms;=2&min;_price=700000&max;_price=2500000

Authorization: Bearer

Example response:

json

```
{
  "success": true,
  "data": [
    {
      "id": 12,
      "title": "Luxury Apartment",
      "score": 91.3,
      "price": 1500000,
      "bedrooms": 2,
      "location": "Nasr City"
    }
  ],
}
```

```
"message": "Search results retrieved",
"errors": null,
"meta": {
  "count": 1,
  "page": 1,
  "pages": 1
}
```

18.11 Get favorites

Endpoint:

- GET /api/v1/favorites/

Example request:

http

GET /api/v1/favorites/

Authorization: Bearer

Example response:

json

```
{
  "success": true,
  "data": [
    {
      "id": 7,
      "property": 12,
      "title": "Luxury Apartment",
      "price": 1500000
    }
  ],
  "message": "Favorites retrieved",
  "errors": null,
  "meta": {
    "count": 1
  }
}
```

18.12 Toggle favorite

Endpoint:

- POST /api/v1/favorites//toggle/

Example request:

http

POST /api/v1/favorites/12/toggle/

Authorization: Bearer

Example response:

json

```
{
  "success": true,
  "data": {
    "property_id": 12,
    "is_favorited": true,
    "action": "added"
  },
  "message": "Favorite updated",
  "errors": null,
  "meta": {}
}
```

18.13 Create inquiry

Endpoint:

- POST /api/v1/inquiries/

Example request:

json

```
{
  "property_id": 12,
  "name": "Ali",
  "phone": "01012345678",
  "message": "I want to visit this apartment this week.",
  "inquiry_type": "visit",
  "preferred_contact_method": "whatsapp",
  "preferred_time": "2026-09-10T18:00:00Z"
}
```

Example response:

json

```
{
  "success": true,
  "data": {
    "id": 22,
    "property": 12,
    "name": "Ali",
    "phone": "01012345678",

```

```
"status": "new",
"agent_notified": false
},
"message": "Inquiry submitted successfully",
"errors": null,
"meta": {}
}
```

18.14 My inquiries

Endpoint:

- GET /api/v1/inquiries/my/

Example response:

json

```
{
"success": true,
"data": [
{
"id": 22,
"property": 12,
"status": "new",
"phone": "01012345678",
"message": "I want to visit this apartment this week."
}
],
"message": "Inquiries loaded",
"errors": null,
"meta": {
"count": 1
}
}
```

18.15 Get inquiry detail

Endpoint:

- GET /api/v1/inquiries//

Example response:

json

```
{
"success": true,
"data": {
"id": 22,
```

```
"property": 12,  
"user": 1,  
"name": "Ali",  
"phone": "01012345678",  
"status": "new",  
"message": "I want to visit this apartment this week."  
},  
"message": "Inquiry detail loaded",  
"errors": null,  
"meta": {}  
}
```

18.16 Developer login

Endpoint:

- POST /api/v1/developer/login/

Example request:

json

```
{  
"email": "developer@example.com",  
"password": "secret-password"  
}
```

Example response:

json

```
{  
"success": true,  
"data": {  
"token": "dev_token_value",  
"user": {  
"id": 4,  
"email": "developer@example.com",  
"full_name": "Developer Team"  
}  
},  
"message": "Login successful",  
"errors": null,  
"meta": {}  
}
```

18.17 Developer register

Endpoint:

- POST /api/v1/developer/register/

Example request:

json

```
{  
  "email": "developer@example.com",  
  "password": "secret-password",  
  "company_name": "Prime Estates"  
}
```

Example response:

json

```
{  
  "success": true,  
  "data": {  
    "id": 4,  
    "email": "developer@example.com",  
    "company_name": "Prime Estates"  
  },  
  "message": "Developer account created",  
  "errors": null,  
  "meta": {}  
}
```

18.18 List developer projects

Endpoint:

- GET /api/v1/developer/projects/

Example response:

json

```
{  
  "success": true,  
  "data": [  
    {  
      "id": 5,  
      "name": "Nile Heights",  
      "status": "active",  
      "location": "Cairo"  
    }  
  ],  
  "message": "Projects retrieved",  
  "errors": null,
```

```
"meta": {  
  "count": 1  
}
```

18.19 Create developer project

Endpoint:

- POST /api/v1/developer/projects/

Example request:

json

```
{  
  "name": "Nile Heights",  
  "description": "A luxury residential project.",  
  "status": "active",  
  "location": "Cairo",  
  "project_type": "residential"  
}
```

Example response:

json

```
{  
  "success": true,  
  "data": {  
    "id": 5,  
    "name": "Nile Heights",  
    "status": "active",  
    "location": "Cairo"  
  },  
  "message": "Project created successfully",  
  "errors": null,  
  "meta": {}  
}
```

18.20 Dashboard overview

Endpoint:

- GET /api/v1/developer/dashboard/overview/

Example response:

json

```
{  
  "success": true,  
  "data": {
```

```
"total_projects": 12,  
"active_projects": 8,  
"total_leads": 96,  
"last_30_days_leads": 21,  
"source_breakdown": {  
  "website": 40,  
  "social": 20  
},  
"message": "Dashboard loaded",  
"errors": null,  
"meta": {}  
}
```

18.21 List leads

Endpoint:

- GET /api/v1/developer/leads/

Example response:

json

```
{  
  "success": true,  
  "data": [  
    {  
      "id": 3,  
      "source": "website",  
      "name": "Sara",  
      "email": "sara@example.com"  
    }  
  ],  
  "message": "Leads retrieved",  
  "errors": null,  
  "meta": {"count": 1}  
}
```

18.22 Export leads

Endpoint:

- GET /api/v1/developer/leads/export/

Example response:

- CSV file download

Typical content:

csv

name,email,source

Sara,sara@example.com,website

18.23 Start AI survey

Endpoint:

- POST /api/v1/ai/survey/start/

Example request:

json

```
{  
  "user_id": 1  
}
```

Example response:

json

```
{  
  "success": true,  
  "data": {  
    "session_id": "sess_abc",  
    "questions": [  
      {"id": 1, "question": "What type of property are you looking for?"}  
    ]  
  },  
  "message": "Survey started",  
  "errors": null,  
  "meta": {}  
}
```

18.24 Answer AI question

Endpoint:

- POST /api/v1/ai/survey/answer/

Example request:

json

```
{  
  "session_id": "sess_abc",  
  "question_id": 1,  
  "answer": "apartment"  
}
```

Example response:

json

```
{
  "success": true,
  "data": {
    "next_question": {
      "id": 2,
      "question": "What is your budget range?"
    }
  },
  "message": "Answer recorded",
  "errors": null,
  "meta": {}
}
```

18.25 Get AI survey results

Endpoint:

- GET /api/v1/ai/survey//results/

Example response:

json

```
{
  "success": true,
  "data": {
    "session_id": "sess_abc",
    "matches": [
      {
        "property_id": 12,
        "title": "Luxury Apartment",
        "score": 91.3,
        "reason": "Matches your budget and preferred type"
      }
    ]
  },
  "message": "Results ready",
  "errors": null,
  "meta": {}
}
```

18.26 Get locations

Endpoint:

- GET /api/v1/locations/

Example response:

```
json
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "Cairo",
      "parent": null
    },
    {
      "id": 2,
      "name": "Nasr City",
      "parent": 1
    }
  ],
  "message": "Locations retrieved",
  "errors": null,
  "meta": {}
}
---
```

19. Overall project architecture summary

This backend is designed with a service-oriented, modular structure:

- models handle persistence
- selectors handle data retrieval
- services encapsulate business logic
- serializers handle validation and output formatting
- views respond to the HTTP layer
- URLs bridge user requests to view methods
- tasks perform async work in the background
- tests cover important flows

This architecture is very appropriate for a growing real-estate backend because it keeps concerns separated and easy to modify.

19. Strengths of the codebase

- clear separation of responsibilities by app
- consistent naming and structure across modules
- shared infrastructure in apps/common/
- strong use of service-layer patterns

- well-defined validation and response handling
- support for OTP verification, search engine logic, dashboard metrics, and AI recommendations
- modular test coverage across different app domains

20. Improvement opportunities

- add stronger upload validation and file checks
- unify authorization patterns further across all developer endpoints
- add more explicit background job monitoring and retry logic
- expand analytics and reporting features
- formalize API versioning and front-end field contract documentation

21. Final conclusion

This project is a complete backend system for a property marketplace. It includes buyer auth, property listing logic, search and ranking, lead capture, developer account operations, project management, dashboard analytics, and AI recommendation flow. The codebase is professionally structured and follows a clean layered architecture that allows it to scale and evolve.

The system is not just a set of disconnected models; it is a fully integrated backend that connects customer behavior, developer operations, and recommendation-based product logic into one platform.